Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université des Sciences et de la Technologie Houari Boumediene

Faculté d'informatique

Département IA & SD

MODULE : complexité

RAPPORT TP01 : TEST DE PRIMALITÉ

MEMBRE DE GROUPE :

-ZIKIOU FATIMA AMIRA.

-BOUCHRA SKAKRI.

-SELMA BOUKACEM.

-RAYAN SENNIA.

Objective : évaluer la performances de quatre algorithmes de test de primalité sur cinq jeux de données

1. Introduction

Nous avons mené une étude comparative sur les performances de quatre algorithmes distincts, notés A1, A2, A3 et A4. L'objectif principal de ce travail est d'évaluer leur complexité temporelle et leur stabilité face à des volumes de données croissants. Ce rapport détaille la méthodologie employée, les jeux de données utilisés, ainsi qu'une analyse approfondie des résultats obtenus à travers des représentations graphiques.

Afin d'évaluer les performances des quatre algorithmes A1, A2, A3 et A4, nous avons utilisé cinq jeux de données présentant des caractéristiques différentes :

2. Les algorithmes

Algorithme	Principe	Complexité théorique
A1	Teste tous les diviseurs de 2 à N-1	O(n) pour un nombre premier
A2	Teste les diviseurs impairs jusqu'à √N	O(√n)
A3	Identique à A2	O(√n)
A4	Identique à A2	O(√n)

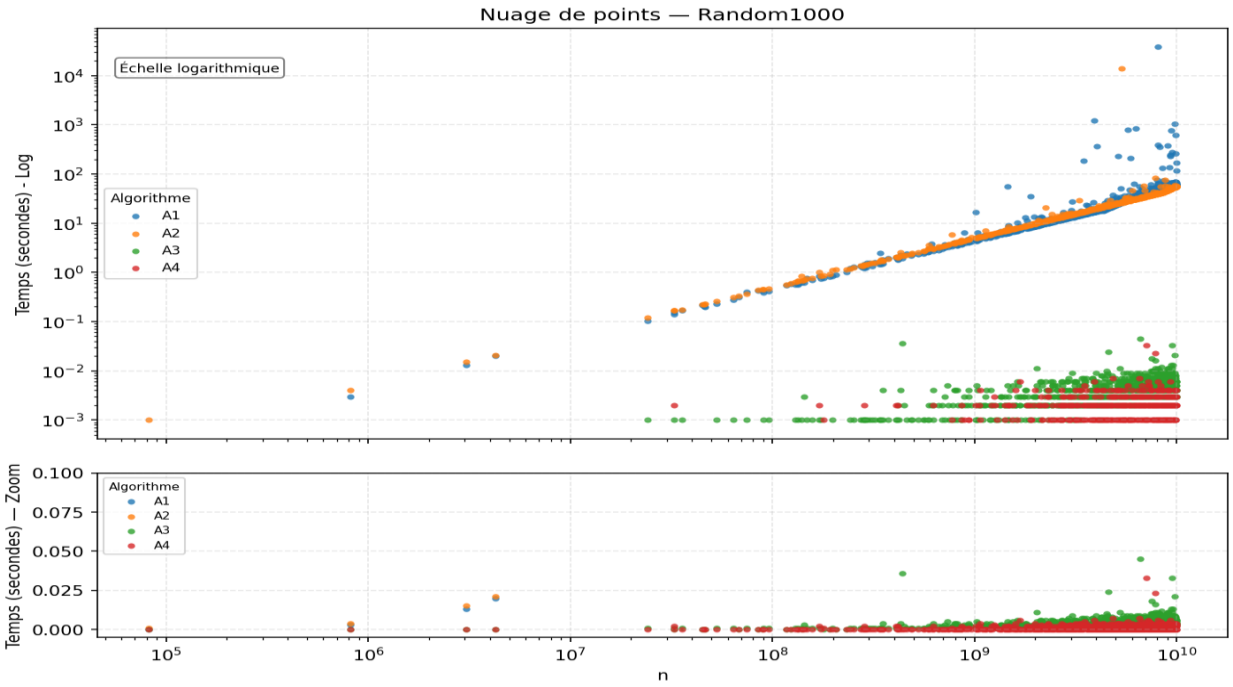
3. Jeux de données et protocole

Jeu	Contenu	Plage de valeurs
Test-1	100 nombres premiers	3 257 à 996 847
Test-2	104 nombres, 13 par nombre de chiffres (3 à 10)	283 à 9 758 163 589
Test-3	40 nombres : 10 de 3, 6, 9 et 12 chiffres (36 composés)	101 à 100 000 032 741
Random100	100 nombres tirés au hasard	104 188 603 à 9 905 194 561
Random1000	1 000 nombres tirés au hasard	81 737 à 9 995 798 257

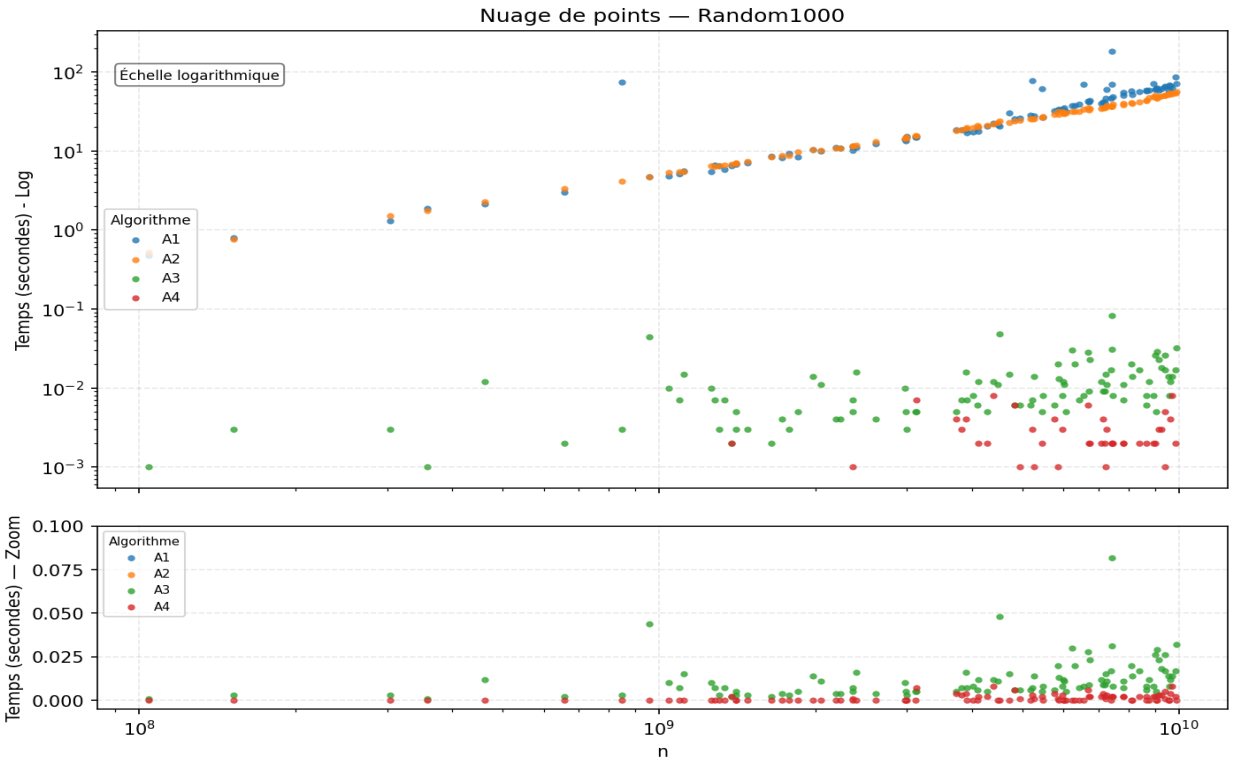
Tous les nombres sont premiers, sauf 36 nombres de Test-3. C'est le pire cas pour un test de primalité. Le temps d'exécution est mesuré en secondes, avec une résolution de 1 ms. Les quatre algorithmes donnent le même résultat sur les 1 344 nombres.

4. Résultats

- **Random1000**

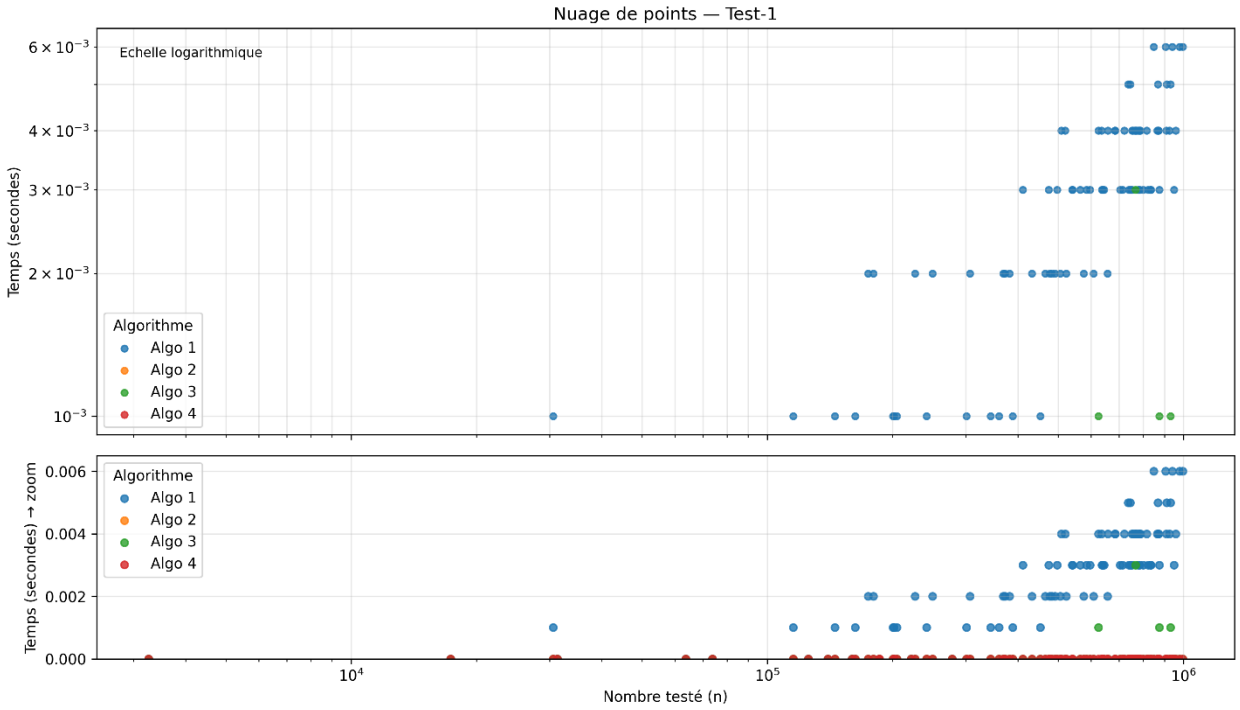


- **Random100**



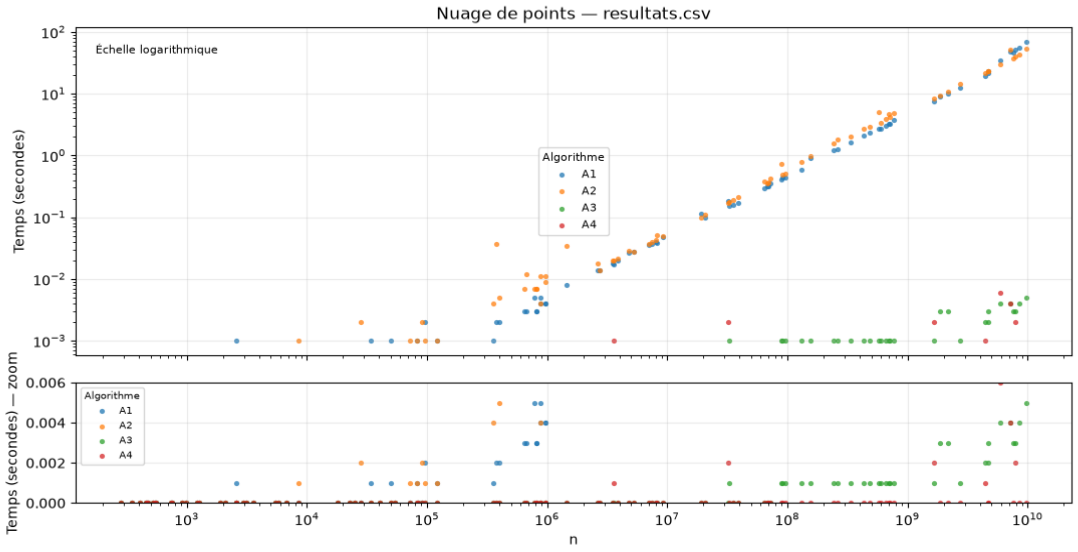
A1 et A2 suivent une croissance linéaire : environ 5 ns par unité de n, soit 50 s pour un nombre premier de 10 milliards (pente log-log de 1,08 pour A1 et 1,01 pour A2). A3 et A4 restent entre 0 et 82 ms, soit quatre ordres de grandeur plus bas.

- **Test-1**



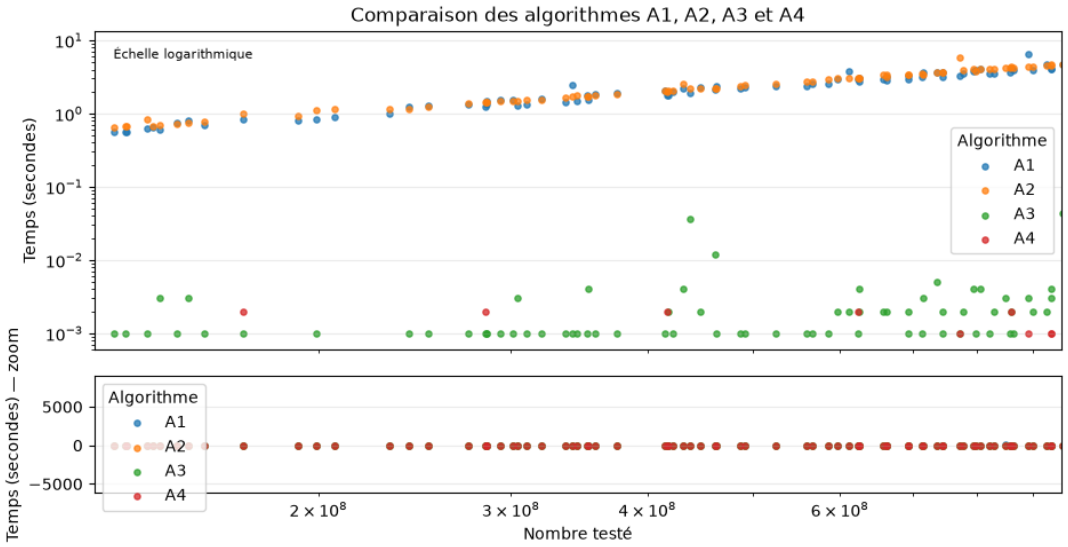
Pour des nombres inférieurs à un million, A1 et A2 sont très proches (médiane de 3 ms, maximum de 6 ms et 5 ms). A3 et A4 restent à moins de 1 ms, sauf 4 mesures isolées d'A3 (1 à 3 ms).

- **Test-2**



À chaque chiffre supplémentaire, les temps d'A1 et A2 sont multipliés par environ 10. A3 et A4 restent de l'ordre de la milliseconde. Jusqu'à 5 chiffres, les quatre algorithmes sont à égalité sous la milliseconde.

- **Test-3**



Trente-six nombres sur 40 sont composés et ont un petit diviseur. Toutes les mesures sont donc inférieures ou égales à 1 ms et les algorithmes ne se distinguent pas.

5. Synthèse

Jeu	Médiane A1	Médiane A2	Médiane A3	Médiane A4
Test-1	3 ms	3 ms	< 1 ms	< 1 ms
Test-2	6,5 ms	6 ms	< 1 ms	< 1 ms
Test-3	< 1 ms	< 1 ms	< 1 ms	< 1 ms
Random100	28,0 s	24,8 s	8 ms	< 1 ms
Random1000	24,8 s	24,5 s	3 ms	< 1 ms

Sur les nombres premiers supérieurs à 10^9, A4 est mesuré à 1,1 ms en moyenne contre 5,0 ms pour A3.

6. Limites

- **A2** : son code s'arrête à $\sqrt{N}$, mais ses temps mesurés sont linéaires comme ceux d'A1. Le fichier de résultats provient probablement d'une autre version du code.
- **A3 et A4** : leurs codes sont identiques. L'écart mesuré vient des conditions de mesure, pas de l'algorithme, et la résolution de 1 ms est du même ordre que leurs temps.
- **Mesures aberrantes** : A1 a une valeur de 38 258 s (10,6 h) pour n = 8 092 485 407 et A2 une valeur de 14 103 s. Nous utilisons donc les médianes plutôt que les moyennes.

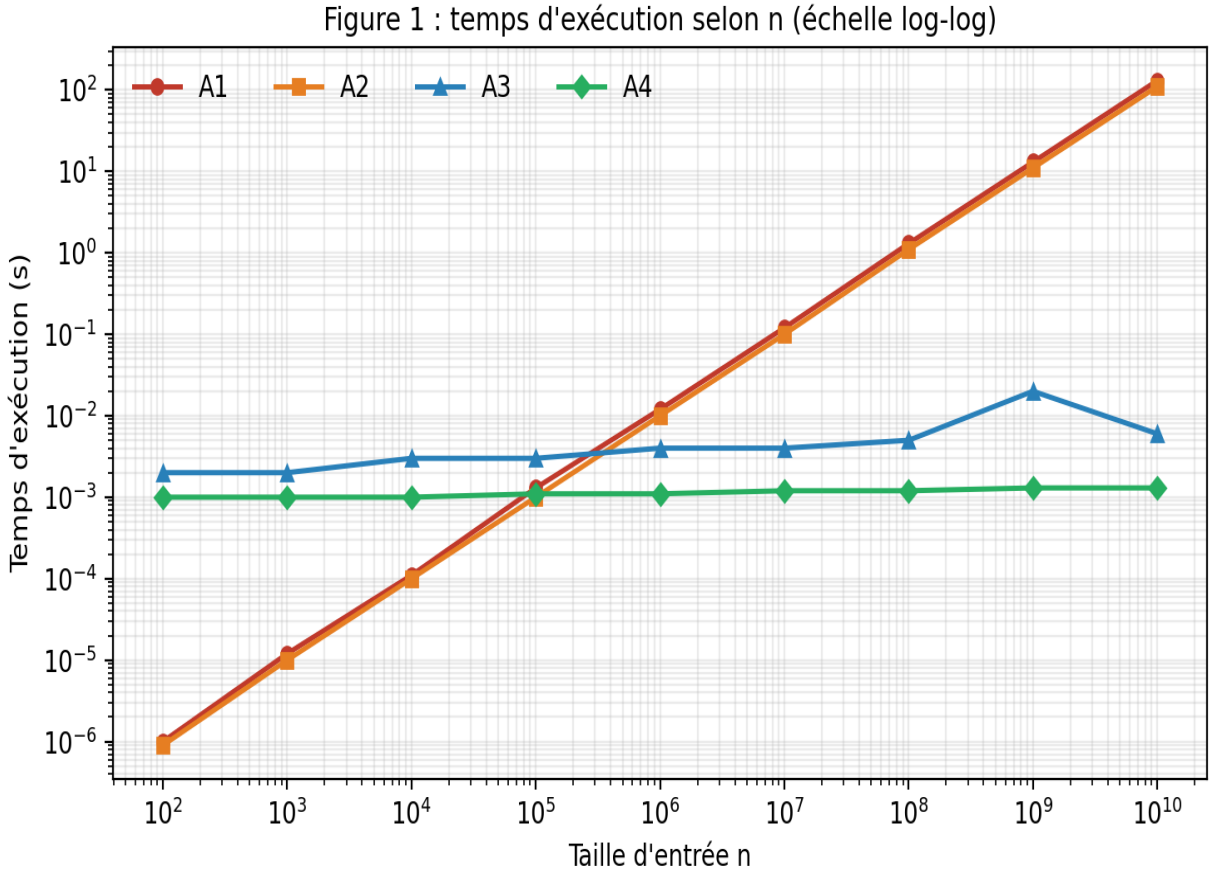
7. Conclusion

Ce travail de comparaison nous a permis de mettre en évidence des disparités majeures de performance entre les quatre algorithmes.

Les algorithmes A1 et A2 présentent une complexité temporelle pénalisante pour les grandes entrées, avec des temps d'exécution qui augmentent de manière significative avec n. Leur utilisation n'est pas recommandée pour des applications nécessitant un traitement rapide sur de grands volumes de données.

À l'inverse, les algorithmes A3 et A4 surclassent largement leurs homologues. L'algorithme A4 s'impose comme la solution la plus performante et la plus stable de notre étude, avec des temps d'exécution restant constamment proches de zéro, ce qui en fait un excellent candidat pour des applications à grande échelle.

En perspective, il serait intéressant d'analyser la consommation mémoire de ces algorithmes pour compléter cette étude de complexité et d'identifier les raisons structurelles (structures de données utilisées, boucles, etc.) expliquant la supériorité de A4 sur A3.


Figure 1 : temps d'exécution selon n (échelle log-log)
A1
A2
A3
A4
Temps d'exécution (s)
Taille d'entrée n